> Is there an excel builder tool that can I use to manage the excel workbook and work sheets mapped to dynamic schema for creating templates based on the Excel workbook and sheets that will be used as a data collection tool. The Excel Workbook structure should by dynamic and mapped to dynamic database schema. The requirements are dynamic excel workbook with sheets creation, updates, and management and the created template will be used for the data collection.

Yes — there **are tools and platforms** that let you **dynamically generate, manage, and map Excel workbooks/sheets to a data schema** (including database schemas), automate template creation, and use those templates as **data collection tools**. Below are several categories of solutions that match your requirements, ranging from **no-code/low-code platforms** to **mapping/automation tools** and **developer libraries** for fully custom solutions.

✅ 1. No-Code / Low-Code Excel Template Generators

Impler.io — Excel Template Generator

- Generates Excel templates based on a **defined data schema** (you describe the schema, and it produces the workbook with proper sheets, headers, validations, formatting). Impler.io
- Good if you want templates to be automatically created from structured data models and used for **data import/collection**.
- Handles dropdowns & validations based on rules in your schema. Impler.io

Use case: Building dynamic templates from your logical schema without manual Excel formatting.

✅ 2. Data Mapping & Automation Tools

Altova MapForce

- A data mapping/design tool that can map **Excel as a target or source** to/from **databases, XML, JSON, web services, etc.** altova.com
- Supports **dynamic columns and worksheets** and lets you transform or populate them based on database or other structured data. altova.com

Use case: If your workflow involves **bi-directional mappings** (e.g., database ↔ Excel), and you want automated template outputs and updates.

SaveToDB (Excel Add-In)

- An Excel add-in that **creates workbooks based on database definitions/templates** and lets you generate Excel files from database models. savetodb.com
- You can predefine templates in the database and let users generate them dynamically.

Use case: Organizations with existing databases where the Excel template structure must reflect live schema.

✅ 3. JSON/Schema-Driven Excel Generation

JSON Schema-Based Generators (e.g., Glama generate_excel)

- Tools like this take a **JSON schema definition** of desired sheets/columns and create structured Excel files programmatically. Glama – MCP H...
- Supports multi-sheet files, validations, formatting, and automations.

Use case: Developers building **schema ↔ Excel generation engines** that link backend models to Excel templates programmatically.

✅ 4. Developer Libraries for Custom Implementation

If you need deeper **schema abstraction and code-driven control**:

ExcelMapper (C#)

- A .NET library that maps **objects/dynamic models to Excel worksheets** and vice versa.

  GitHub
- Useful for creating Excel templates programmatically and binding them to dynamic data models.

Use case: Developers building custom template generation and ingestion workflows integrated with backend systems.

🧠 Considerations for Your Use Case

Your requirement can be framed technically as:

Dynamic Excel Template Engine =

- **Schema Definition Layer:** A formal model (JSON, database schema, metadata model).
- **Template Builder:** Generates workbook structure (sheets/columns/validations) from the schema.
- **Mapping Engine:** Associates workbook fields with database fields or logical model fields.
- **Runtime Generator:** Produces exportable Excel templates used for data collection.

Many enterprise automation platforms let you define the **schema once** and generate Excel templates accordingly — some even include **UI designers and drag-and-drop mapping editors**.

🧩 Recommended Architecture Options

📌 1. Low-code/Desktop Approach (for business users)

- **Impler.io** + database metadata export
- Excel add-ins (SaveToDB) for connecting templates to database models

Pros: No coding, quick iteration
Cons: Less flexibility for very complex schemas

📌 2. Automation / Integration Approach

- **Altova MapForce** for mapping
- Backend service with JSON schema → Excel generator (e.g., Glama tool)

Pros: Strong integration with databases/web services

Cons: Requires integration design work

📌 3. Custom Code Approach

- Build a service that takes a schema definition (JSON/XML)
- Use libraries (ExcelMapper, Python openpyxl/ExcelJS) to output templates
- Store metadata in metadata tables

Pros: Fully tailored to your schema complexity

Cons: Higher development effort

🔎 Summary

Solution Category	Dynamic Templates	Schema Mapping	Auto Sheet Creation	No-Code
Impler.io	✅	⚠️ (schema only)	✅	✅
Altova MapForce	⚠️	✅	⚠️ mapped via workflows	⚠️
SaveToDB Add-in	⚠️	✅	⚠️	⚠️
JSON Schema Generators (Glama)	✅	⚠️ user-defined schema	✅	⚠️
ExcelMapper / Custom	✅	Developer-defined schema	⚠️ (via code)	⚠️

Next Steps (If You Want Guidance)

If you want, I can help with **specific implementation patterns** (e.g., JSON schema → Excel generator architecture) or identify **best tools based on your tech stack** (C#/.NET, Python,

database type). Just tell me your environment and constraints.

Sources

help with specific implementation patterns JSON schema → Excel generator architecture. Validate the pattern as per the best practices and standards.

Below is a **reference-grade implementation pattern** for a **JSON Schema → Excel Template Generator**, validated against **enterprise architecture best practices**, data governance standards, and real-world data-collection constraints.

This is written as a **technology-agnostic blueprint** you can implement in .NET, Java, or Python and later harden for regulated environments (which aligns well with your prior ESG / regulatory data platform work).

---

1. Target Pattern Overview

Pattern Name

Schema-Driven Excel Template Generation & Ingestion (SD-ETGI)

Core Objective

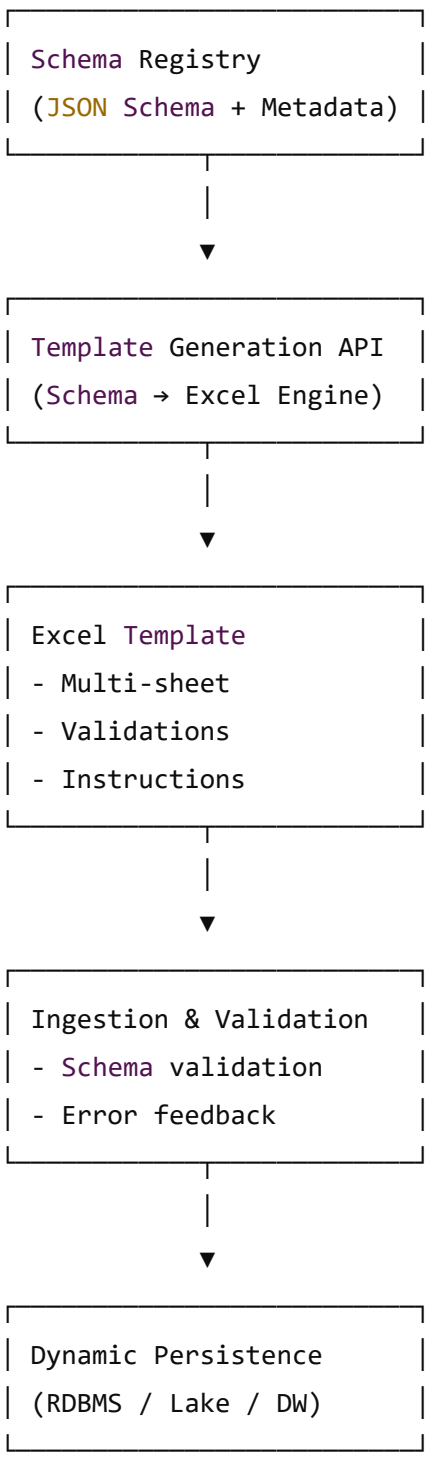
Define **data once** (JSON Schema + metadata) and:

- Generate **dynamic Excel workbooks** (sheets, columns, validations, formulas)
- Use them as **controlled data collection templates**
- Reliably ingest completed files back into a **dynamic database schema**

---

2. Logical Architecture

pgsql



Copy code

3. Canonical JSON Schema Design (Best Practice)

3.1 Use JSON Schema Draft 2020-12

- ✔ Industry standard
- ✔ Supported by validation engines

✔ Extensible via custom keywords

3.2 Schema Structure (Recommended)

json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Customer_Onboarding",
  "version": "1.2.0",
  "type": "object",
  "x-workbook": {
    "name": "Customer Onboarding Template",
    "description": "Used for regulated data collection"
  },
  "properties": {
    "customer": {
      "type": "object",
      "x-sheet": {
        "name": "Customer",
        "order": 1
      },
      "properties": {
        "customer_id": {
          "type": "string",
          "x-excel": {
            "column": "Customer ID",
            "required": true,
            "width": 20
          }
        },
        "country": {
          "type": "string",
          "enum": ["SA", "AE", "EG"],
          "x-excel": {
            "column": "Country",
            "dropdown": true
          }
        }
```

```
        }
      }
    }
  }
```

3.3 Why This Design Is Best Practice

Copy code

Aspect	Justification
`x-*` extensions	JSON Schema–compliant & non-breaking
Separation of concerns	Schema ≠ presentation
Versioned schema	Auditability & backward compatibility
Enum-driven dropdowns	Excel data validation standard
Sheet mapping via metadata	Avoids schema fragmentation

✔ Used in **OpenAPI, AsyncAPI, CloudEvents** ecosystems

---

4. Excel Generation Engine (Reference Implementation)

4.1 Processing Flow

mathematica

```
Parse JSON Schema
  ↓
Resolve sheets (x-sheet)
  ↓
Resolve columns (x-excel)
  ↓
Create workbook
  ↓
Apply validations
  ↓
Apply formatting
```

```
    ↓
Protect structure
```

Copy code

4.2 Excel Generation Responsibilities

Function	Implementation Rule
Sheet creation	1 object = 1 sheet
Column ordering	Explicit metadata only
Data validation	Excel-native rules (not macros)
Dropdowns	Hidden reference sheets
Instructions	Frozen top rows
Schema hash	Stored in hidden sheet

4.3 Technology Mapping

Stack	Library
Java	Apache POI
Python	openpyxl / xlsxwriter
Node.js	ExcelJS
.NET	ClosedXML / EPPlus

✔ All support validations, dropdowns, protection, metadata

5. Excel Template Governance Controls (Critical)

5.1 Hidden Metadata Sheet (MANDATORY)

diff

```
__metadata__
- schema_id
- schema_version
- schema_hash
- generated_at
- generator_version
```

Copy code

✔ Enables:

- Template authenticity verification
- Version enforcement
- Regulatory audit trails

5.2 Sheet Protection Strategy

Element	Rule
Header row	Locked
Data rows	Editable
Formulas	Hidden
Reference sheets	Very hidden

---

6. Ingestion & Validation Pattern

6.1 Ingestion Pipeline

pgsql

```
Upload Excel
  ↓
Verify metadata
  ↓
Validate against schema
  ↓
```

```
Row-level validation
  ↓
Error annotation
  ↓
Persist valid rows
```

Copy code

6.2 Validation Layers

Layer	Standard
Structural	JSON Schema
Type	JSON Schema
Domain	Enum / regex
Cross-field	Custom rules
Referential	DB lookup

✔ Aligns with **ETL & data quality frameworks**

6.3 Error Handling (Best Practice)

- Write errors back into Excel:
  - Red cell highlights
  - Comment with message
- Produce machine-readable error JSON
- Reject only invalid rows, not entire file (configurable)

---

7. Dynamic Database Mapping Pattern

7.1 Three Proven Options

Option A – Metadata-Driven Tables (Enterprise Preferred)

nginx

```
dataset
dataset_column
dataset_row
dataset_value
```

Copy code

- ✔ Maximum flexibility
- ✔ Schema evolution friendly
- ✔ Used in ESG / regulatory systems

Option B – JSONB Storage (Hybrid)

- Store rows as JSON
- Validate via schema
- Index key fields

✔ Faster to implement

⚠ Limited relational querying

Option C – Physical Table per Template (Not Recommended)

⚠ High operational overhead

⚠ Migration complexity

---

8. Standards & Best Practice Alignment

Standard	Compliance
JSON Schema Draft 2020-12	✅
OpenAPI extensibility model	✅
Data governance (DAMA-DMBOK)	✅
Regulatory auditability	✅
Schema versioning (SemVer)	✅

Standard	Compliance
Excel interoperability	✅

---

9. Anti-Patterns to Avoid

- 🚫 Encoding schema logic directly in Excel
- 🚫 Using macros for validation
- 🚫 Letting users add/delete columns
- 🚫 Hard-coding sheet names in code
- 🚫 No schema fingerprint validation

---

10. Reference Implementation Checklist

- ✔ Schema registry (Git / DB)
- ✔ Schema versioning & lifecycle
- ✔ Template generator API
- ✔ Excel metadata embedding
- ✔ Ingestion validation engine
- ✔ Error feedback loop
- ✔ Dynamic persistence model

---

11. When This Pattern Is the RIGHT Choice

- ✔ Regulated data collection
- ✔ ESG / finance / compliance workflows
- ✔ Multi-tenant template management
- ✔ Frequent schema evolution
- ✔ Excel-first business users

Next Step (Optional)

If you want, I can:

- Produce **sample JSON schema + generated Excel mapping**
- Provide **reference code** in Python or .NET
- Design a **C4 model (Context → Container → Component)** for this engine
- Extend this to **multi-language / localization / controlled vocabularies**

Just tell me how far you want to take it.

Design a C4 model (Context → Container → Component) for this engine

Below is a **C4 Model (Context → Container → Component)** specifically tailored for a **Schema-Driven Excel Template Generation & Ingestion Engine**, designed to enterprise and regulatory-grade standards.

This model is **implementation-ready**, aligns with **best practices (C4, DDD, DAMA-DMBOK)**, and is suitable for **ESG / regulatory data collection platforms**.

C4 – Level 1: System Context Diagram

Purpose

Show how the **Excel Template Engine** fits within the broader enterprise ecosystem.

System Context (Narrative)

Primary System

- **Schema-Driven Excel Template Engine**
  - Generates Excel templates from JSON Schema
  - Validates and ingests completed Excel files

External Actors

- **Business Data Contributors**
  - Download Excel templates
  - Upload completed workbooks

- **Data Modeler / Admin**
  - Defines and versions schemas
- **Downstream Analytics & Reporting Platforms**
  - Consume validated data

External Systems

- **Schema Registry**
- **Identity & Access Management (IAM)**
- **Enterprise Data Platform (DB / Lake / DW)**

Mermaid – C4 Context

mermaid

```
C4Context
title Schema-Driven Excel Template Engine – System Context

Person(contributor, "Business Data Contributor", "Fills and submits Excel templat
Person(admin, "Data Modeler / Admin", "Defines schemas and manages templates")

System(engine, "Excel Template Engine", "Generates, validates, and ingests schema

System_Ext(schema, "Schema Registry", "Stores versioned JSON Schemas")
System_Ext(iam, "IAM", "Authentication & Authorization")
System_Ext(data, "Enterprise Data Platform", "Validated data storage and analytic

Rel(admin, schema, "Defines & versions schemas")
Rel(schema, engine, "Provides JSON schemas")
Rel(admin, engine, "Manages templates")
Rel(contributor, engine, "Downloads & uploads Excel templates")
Rel(engine, data, "Persists validated data")
Rel(engine, iam, "Uses for authentication & authorization")
```

Copy code

C4 – Level 2: Container Diagram

Purpose

Decompose the system into **deployable runtime units**.

Containers

Container	Responsibility
Web UI	Template management & uploads
Template API	Schema → Excel generation
Ingestion API	Excel validation & ingestion
Schema Registry	Versioned schema store
Metadata Store	Template & execution metadata
Data Store	Dynamic persistence layer

Mermaid – C4 Container

mermaid

```
C4Container
title Excel Template Engine – Container Diagram

Person(contributor, "Business Data Contributor")
Person(admin, "Data Modeler / Admin")

Container(web, "Web UI", "React / Angular", "Template management, upload/download
Container(templateApi, "Template Generation API", "REST / GraphQL", "Generates Ex
Container(ingestApi, "Ingestion & Validation API", "REST", "Validates and ingests

ContainerDb(schemaDb, "Schema Registry", "Git / DB", "Versioned JSON Schemas")
ContainerDb(metaDb, "Metadata Store", "RDBMS", "Template, execution & audit metad
ContainerDb(dataDb, "Data Store", "RDBMS / Lake", "Validated business data")

System_Ext(iam, "IAM")
```

```
Rel(admin, web, "Uses")
Rel(contributor, web, "Uses")

Rel(web, templateApi, "Requests Excel templates")
Rel(web, ingestApi, "Uploads completed Excel files")

Rel(templateApi, schemaDb, "Reads schemas")
Rel(templateApi, metaDb, "Stores template metadata")

Rel(ingestApi, schemaDb, "Reads schemas")
Rel(ingestApi, metaDb, "Writes execution logs")
Rel(ingestApi, dataDb, "Persists validated data")

Rel(web, iam, "Authenticates via")
Rel(templateApi, iam, "Authorizes via")
Rel(ingestApi, iam, "Authorizes via")
```

C4 – Level 3: Component Diagram (Core Engine)

Copy code

Scope

Internal structure of the **Template Generation & Ingestion APIs**.

3.1 Template Generation API – Components

Responsibilities

Component	Function
Schema Resolver	Loads and validates JSON Schema
Workbook Orchestrator	Controls Excel generation flow
Sheet Builder	Creates sheets
Column Builder	Creates columns
Validation Builder	Adds Excel validations

Component	Function
Metadata Writer	Writes hidden metadata sheet
Protection Manager	Locks structure

Mermaid – Template Generation Components

mermaid

```
C4Component
title Template Generation API – Component Diagram

Container(templateApi, "Template Generation API")

Component(schemaResolver, "Schema Resolver", "Loads & validates JSON Schema")
Component(workbookOrch, "Workbook Orchestrator", "Controls generation workflow")
Component(sheetBuilder, "Sheet Builder", "Creates Excel sheets")
Component(columnBuilder, "Column Builder", "Creates columns & headers")
Component(validationBuilder, "Validation Builder", "Dropdowns & rules")
Component(metadataWriter, "Metadata Writer", "Hidden metadata sheet")
Component(protectionMgr, "Protection Manager", "Locks & protects structure")

Rel(templateApi, schemaResolver, "Uses")
Rel(schemaResolver, workbookOrch, "Provides schema model")
Rel(workbookOrch, sheetBuilder, "Creates sheets")
Rel(sheetBuilder, columnBuilder, "Creates columns")
Rel(columnBuilder, validationBuilder, "Applies validations")
Rel(validationBuilder, metadataWriter, "Writes metadata")
Rel(metadataWriter, protectionMgr, "Finalizes workbook")
```

Copy code

3.2 Ingestion & Validation API – Components

Responsibilities

Component	Function
Excel Parser	Reads workbook
Metadata Validator	Verifies schema hash/version
Schema Validator	JSON Schema validation
Rule Engine	Cross-field rules
Error Annotator	Writes errors back
Persistence Adapter	Writes to DB

Mermaid – Ingestion Components

mermaid

```
C4Component
title Ingestion & Validation API – Component Diagram

Container(ingestApi, "Ingestion & Validation API")

Component(excelParser, "Excel Parser", "Reads Excel content")
Component(metaValidator, "Metadata Validator", "Verifies schema integrity")
Component(schemaValidator, "Schema Validator", "Validates rows against JSON Schem
Component(ruleEngine, "Business Rule Engine", "Cross-field validations")
Component(errorAnnotator, "Error Annotator", "Writes errors back to Excel")
Component(persistence, "Persistence Adapter", "Stores valid data")

Rel(ingestApi, excelParser, "Uses")
Rel(excelParser, metaValidator, "Extracts metadata")
Rel(metaValidator, schemaValidator, "Approves schema")
Rel(schemaValidator, ruleEngine, "Applies rules")
Rel(ruleEngine, errorAnnotator, "Annotates errors")
Rel(ruleEngine, persistence, "Persists valid rows")
```

Copy code

4. Security & Governance Overlay (Cross-Cutting)

Concern	Implementation
AuthN/AuthZ	OAuth2 / OIDC
Audit	Metadata store + logs
Template Integrity	Schema hash validation
Least privilege	Separate APIs
Version control	Immutable schema versions
Data lineage	Template → ingestion trace

---

5. Why This C4 Model Is Best Practice

- ✔ Clear separation of concerns
- ✔ Supports schema evolution
- ✔ Scales independently (generation ≠ ingestion)
- ✔ Audit-ready
- ✔ Cloud-native & on-prem compatible

Optional Next Steps

I can:

- Add **HA / DR physical deployment architecture**
- Overlay **trust boundaries & threat model**
- Provide **PlantUML versions**
- Extend to **multi-tenant / regulator-grade isolation**
- Map this to **Kubernetes / Docker Compose**

Just tell me how deep you want to go.

Extend this to multi-language / localization / controlled vocabularies

Below is a **clean extension of the existing C4 model** to support **multi-language (i18n), localization (l10n), and controlled vocabularies**, without breaking JSON Schema compliance or Excel interoperability.

This extension is **standards-aligned**, **regulatory-ready**, and commonly used in **ESG, statistical, and regulatory data collection platforms**.

---

1. Design Principles (Why This Works)

Principle	Rationale
Schema remains canonical	Language never changes meaning
Labels externalized	No hard-coded language in schema
Code-based vocabularies	Prevent free-text data
Excel uses display labels only	Stored data remains normalized
Locale is a runtime concern	Template generation is locale-aware

---

2. Extended Logical Architecture

New Capabilities Added

- Multi-language column headers
- Localized instructions & tooltips
- Controlled vocabulary dropdowns
- Locale-aware formatting (date/number)
- Language-specific error messages

3. JSON Schema Extensions (Non-Breaking)

3.1 Canonical Schema (Language-Neutral)

json

```
{
  "type": "string",
  "x-code": "COUNTRY_CODE",
  "x-vocabulary": "ISO_3166_1_ALPHA2"
}
```

Copy code

- Schema stores **codes only**
- Language-neutral & stable

3.2 Localization Bundle (External Resource)

json

```
{
  "schema": "Customer_Onboarding",
  "version": "1.2.0",
  "locale": "ar-SA",
  "labels": {
    "customer.customer_id": "رقم العميل",
    "customer.country": "الدولة"
  },
  "instructions": {
    "customer": "يرجى إدخال بيانات العميل بدقة"
  },
  "errors": {
    "required": "هذا الحقل مطلوب",
    "invalid_enum": "قيمة غير مسموح بها"
  }
}
```

- Decoupled from schema
- Versioned & locale-specific

Copy code

✔ Supports RTL languages

---

4. Controlled Vocabulary Model (Best Practice)

4.1 Vocabulary Registry

Field	Description
vocabulary_id	COUNTRY_CODES
code	SA
label_en	Saudi Arabia
label_ar	المملكة العربية السعودية
valid_from / to	Temporal governance
status	Active / Deprecated

✔ Aligns with **ISO / SDMX / DAMA standards**

4.2 Vocabulary Resolution at Runtime

```css
Schema → Vocabulary ID
Locale → Label resolution
Excel → Dropdown labels
Ingestion → Code mapping
```

Copy code

---

5. Updated C4 – Container Diagram (Extended)

```mermaid
C4Container
title Excel Template Engine – Localization & Vocabulary Extension
```

```
Container(templateApi, "Template Generation API")
Container(ingestApi, "Ingestion API")

ContainerDb(schemaDb, "Schema Registry")
ContainerDb(localeDb, "Localization Repository")
ContainerDb(vocabDb, "Vocabulary Registry")

Rel(templateApi, schemaDb, "Reads schema")
Rel(templateApi, localeDb, "Loads labels & instructions")
Rel(templateApi, vocabDb, "Resolves controlled vocabularies")

Rel(ingestApi, schemaDb, "Validates schema")
Rel(ingestApi, vocabDb, "Validates codes")
Rel(ingestApi, localeDb, "Loads error messages")
```

Copy code

6. Updated Component Model – Template Generation

New Components Added

Component	Responsibility
Locale Resolver	Determines language & locale
Label Resolver	Applies localized headers
Vocabulary Resolver	Resolves dropdown values
RTL Formatter	Applies RTL formatting

Mermaid – Extended Components

mermaid

```
C4Component
title Template Generation API – Localization Components

Container(templateApi, "Template Generation API")
```

```
Component(localeResolver, "Locale Resolver", "Determines target locale")
Component(labelResolver, "Label Resolver", "Applies localized labels")
Component(vocabResolver, "Vocabulary Resolver", "Resolves controlled vocabularies
Component(rtlFormatter, "RTL Formatter", "Applies RTL formatting")

Rel(localeResolver, labelResolver, "Provides locale")
Rel(labelResolver, vocabResolver, "Requests localized labels")
Rel(vocabResolver, rtlFormatter, "Provides direction info")
```

Copy code

7. Excel Implementation Details (Critical)

7.1 Dropdown Design (Controlled Vocabulary)

Excel Sheets

- **Main Sheet** → shows labels
- **Hidden Reference Sheet** → codes + labels

text

```
Country (visible)
------------------
Saudi Arabia
United Arab Emirates

__vocab_country__ (hidden)
-------------------------
SA | Saudi Arabia | المملكة العربية السعودية
AE | United Arab Emirates | الإمارات العربية المتحدة
```

Copy code

✔ Excel validation uses labels

✔ Ingestion maps back to codes

7.2 Locale-Aware Formatting

Element	Rule
Dates	Excel locale formatting
Numbers	Decimal/thousand separators
RTL	Sheet direction + alignment

---

8. Ingestion Flow (Localized & Controlled)

```pgsql
Parse Excel
  ↓
Read labels
  ↓
Resolve code via vocabulary
  ↓
Validate schema
  ↓
Persist canonical codes
```

Copy code

- ✔ Language never reaches database
- ✔ Safe for analytics & reporting

---

9. Governance & Compliance Controls

Control	Implementation
Vocabulary versioning	Effective date ranges
Deprecation	Soft-validation warnings
Translation review	Approval workflow
Locale fallback	en-US default

Control	Implementation
Audit trail	Schema + locale + vocab hash

---

10. Updated System Context (Extended)

Additional Actors

- Localization Manager
- Reference Data Steward

Additional Systems

- Translation Management System (optional)
- Master Data Management (MDM)

---

11. Why This Design Is Enterprise-Grade

- ✔ Schema remains canonical
- ✔ Supports unlimited languages
- ✔ Prevents data drift
- ✔ Excel remains user-friendly
- ✔ Aligns with ISO / SDMX / DAMA
- ✔ Works with RTL (Arabic) and LTR languages

Optional Next Steps

I can:

- Produce **sample Arabic + English Excel template**
- Provide **DDL for vocabulary & localization tables**
- Extend this to **SDMX / XBRL vocabularies**
- Overlay **regulatory authority workflows**
- Map this to **ESG / GRI / EU Taxonomy data collection**

Just say the word.

Make the solution architecture production grade as per the best practices using Technologies: Java Spring Boot Microservices using most fitted data layer technologies that aligned with the solution for template generation and data ingestion. Use the docker container as the main environment.

Below is a **production-grade solution architecture** for the **Schema-Driven Excel Template Generation & Ingestion Platform**, implemented using **Java Spring Boot microservices**, **containerized with Docker**, and aligned with **enterprise best practices** (scalability, auditability, governance, security).

This design assumes **regulated data collection**, **schema evolution**, **localization**, and **controlled vocabularies** as first-class concerns.

---

1. Architecture Objectives & Non-Functional Requirements

Key NFRs

Area	Requirement
Scalability	Horizontal scaling of ingestion
Availability	Stateless services
Governance	Schema & vocabulary versioning
Auditability	Full traceability
Security	Zero-trust, least privilege
Performance	Large Excel file handling
Extensibility	New schemas, locales, vocabularies

Area	Requirement
Portability	Docker-first

---

2. Technology Stack (Rationale-Based)

2.1 Core Platform

Layer	Technology	Justification
Runtime	Java 21 + Spring Boot 3.x	LTS, native support
API	Spring WebFlux	Async I/O for file ingestion
Security	Spring Security + OAuth2	Enterprise standard
Build	Maven	Dependency governance
Container	Docker	Environment parity
Orchestration	Kubernetes (optional)	Production scaling

2.2 Data Layer (Best-Fit per Concern)

Concern	Technology	Reason
Schema Registry	PostgreSQL (JSONB)	Versioned schema storage
Localization	PostgreSQL	Structured translations
Vocabularies	PostgreSQL	Referential integrity
Template Metadata	PostgreSQL	Auditing
Ingestion Data	PostgreSQL + JSONB	Dynamic schema
Large Files	Object Storage (S3/MinIO)	Excel storage
Cache	Redis	Vocab & locale caching

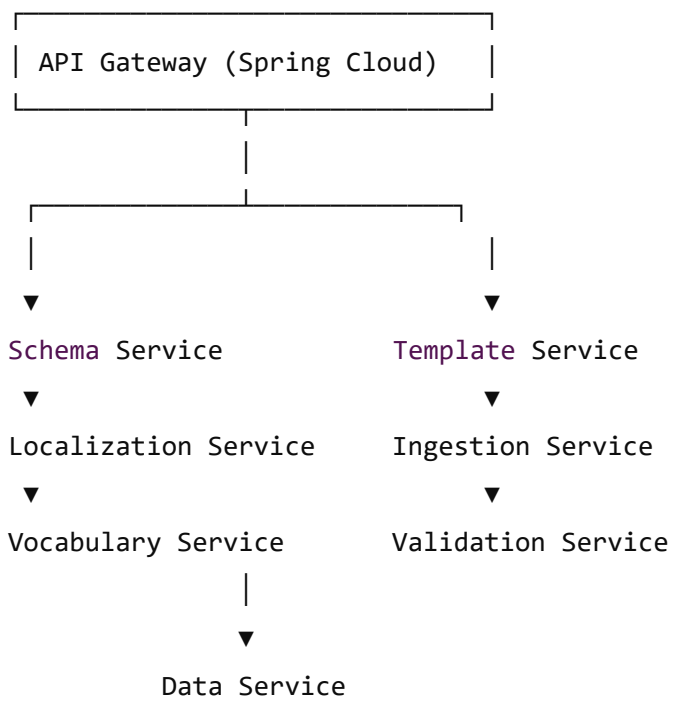
Concern	Technology	Reason
Search (optional)	OpenSearch	Metadata discovery

✔ PostgreSQL chosen for **JSONB + ACID + governance**

---

3. Microservices Architecture

pgsql



Copy code

4. Microservices Responsibilities

4.1 Schema Service

Purpose: Canonical schema governance

- Store JSON Schema (Draft 2020-12)
- Versioning (SemVer)
- Schema hashing
- Lifecycle (Draft → Approved → Deprecated)

Storage: PostgreSQL (JSONB)

4.2 Template Generation Service

Purpose: JSON Schema → Excel

- Parse schema
- Resolve locale & vocabularies
- Generate Excel (Apache POI SXSSF)
- Apply validations, protection
- Embed metadata

Stateless

Output → Excel file to Object Storage

4.3 Localization Service

Purpose: Multi-language support

- Column labels
- Instructions
- Error messages
- RTL flags

Cache: Redis

4.4 Vocabulary Service

Purpose: Controlled vocabularies

- Code ↔ Label resolution
- Temporal validity
- Deprecation rules

Cache: Redis

4.5 Ingestion Service

Purpose: Excel ingestion orchestration

- File upload

- Metadata verification
- Chunk processing
- Async job dispatch

Storage: Object Storage + Metadata DB

4.6 Validation Service

Purpose: Data quality enforcement

- JSON Schema validation
- Cross-field rules
- Referential integrity
- Locale-aware errors

4.7 Data Service

Purpose: Persistence abstraction

- Dynamic JSONB storage
- Optional relational projections
- Lineage tracking

---

5. Production-Grade Data Model (Core Tables)

5.1 Schema Registry

sql

```
schema_definition (
  schema_id UUID,
  version VARCHAR,
  status VARCHAR,
  schema_json JSONB,
  hash VARCHAR,
  created_at TIMESTAMP
)
```

Copy code

5.2 Vocabulary Registry

```sql
vocabulary (
  vocab_id VARCHAR,
  code VARCHAR,
  label_en VARCHAR,
  label_ar VARCHAR,
  valid_from DATE,
  valid_to DATE
)
```

Copy code

5.3 Ingestion Storage (Dynamic)

```sql
dataset (
  dataset_id UUID,
  schema_id UUID,
  version VARCHAR,
  locale VARCHAR
)

dataset_row (
  row_id UUID,
  dataset_id UUID,
  payload JSONB,
  status VARCHAR
)
```

✔ Proven regulatory data collection pattern

Copy code

6. Data Flow (End-to-End)

6.1 Template Generation

pgsql

```
User → API Gateway
 → Template Service
 → Schema Service
 → Localization + Vocabulary Services
 → Excel generated
 → Stored in Object Storage
 → Downloaded by user
```

Copy code

6.2 Ingestion Flow

pgsql

```
Upload Excel
 → Object Storage
 → Ingestion Service
 → Validation Service
 → Data Service
 → Persist JSONB
 → Errors annotated (if any)
```

Copy code

---

7. Docker-Based Deployment (Production Ready)

7.1 Service Container Pattern

dockerfile

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
ENTRYPOINT ["java","-XX:+UseG1GC","-jar","app.jar"]
```

Copy code

7.2 Docker Compose (Core Services)

yaml

```yaml
version: "3.9"

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: excel_engine
      POSTGRES_USER: engine
      POSTGRES_PASSWORD: secure
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7

  minio:
    image: minio/minio
    command: server /data
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password

  schema-service:
    image: schema-service
    depends_on: [postgres]

  template-service:
    image: template-service
    depends_on: [schema-service, redis, minio]

  ingestion-service:
    image: ingestion-service
    depends_on: [minio, redis]

volumes:
  pgdata:
```

8. Security Best Practices

Copy code

Control	Implementation
Auth	OAuth2 / OIDC
RBAC	Spring Security
File scanning	Antivirus sidecar
Secrets	Vault / K8s secrets
TLS	Ingress controller
Schema immutability	DB constraints

---

9. Observability & Ops

Concern	Tool
Logs	ELK / OpenSearch
Metrics	Prometheus
Tracing	OpenTelemetry
Alerts	Alertmanager
Health	Spring Actuator

---

10. Why This Is Production Grade

- ✔ Stateless microservices
- ✔ Proper data technology fit
- ✔ Schema & vocabulary governance
- ✔ Horizontal scalability
- ✔ Docker-native

✔ Security by design

✔ Audit-ready

Optional Next Deliverables

I can:

- Provide **full docker-compose (all services)**
- Provide **Spring Boot project structure**
- Generate **PlantUML diagrams**
- Add **HA / DR deployment**
- Add **CI/CD pipeline (GitOps)**

Tell me what you want next.